KTÜ PRESS

2026–2027

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

WEEK 1

Constraint Satisfaction Problems: A Formal Definition

Constraint Programming

Learning phases: 1 · Pre-instruction 2 · Content 3 · Participation 4 · Assessment 5 · Follow-through

Assoc. Prof. Tolga Berber
KTU Faculty of Science, Department of Computer Science · https://avesis.ktu.edu.tr/tberber

CP

1 · Pre-instruction

Why It Matters?

You have solved a constraint satisfaction problem before without ever calling it one.

- Building a class schedule that avoids double-booking a room
- Filling in a Sudoku grid
- Seating people so that no two who cannot stand each other sit together

Every time, you chose values for unknowns under restrictions – that is a CSP.

This week gives that everyday pattern a name, a shared vocabulary, and – for the rest of the course – a machine that solves it for you.

This Week

◎ Learning Objectives

By the end of this week, you will be able to:

1. Define a CSP (variables, domains, constraints) and distinguish it from a COP and from SAT
2. Explain why modeling is a skill: viewpoints, auxiliary variables, implied constraints
3. Formulate N-queens, Sudoku, map coloring and magic square as CSPs on paper
4. Build and run a first OR-Tools CP-SAT model, including enumerating all solutions
5. Translate a paper CSP model into a verified, running CP-SAT program

UNIT 1

What a CSP Is

2 · Content The Formal Triple

- **Variables** $X_1, \ldots, X_n$ – the unknowns you must decide
- **Domains** D_i – the finite set of values X_i may take
- **Constraints** – restrictions on which combinations of values are jointly allowed

Definition: Constraint Satisfaction Problem

A CSP is a triple (X, D, C); a *solution* is a complete assignment $X_i \mapsto v_i \in D_i$ for every i that satisfies every constraint in C.

UNIT 1 · CSP BASICS

Analogy: A Form with Fine Print

Analogy

Each blank on a form is a variable; the entries it accepts are its domain (a date field only accepts dates); the form's fine print ("field 4 must be later than field 2") is a constraint. A correctly filled-out form is a solution.

A complete assignment fills every blank; a solution additionally obeys every rule in the fine print – the two are not the same thing.

UNIT 1 · CSP BASICS

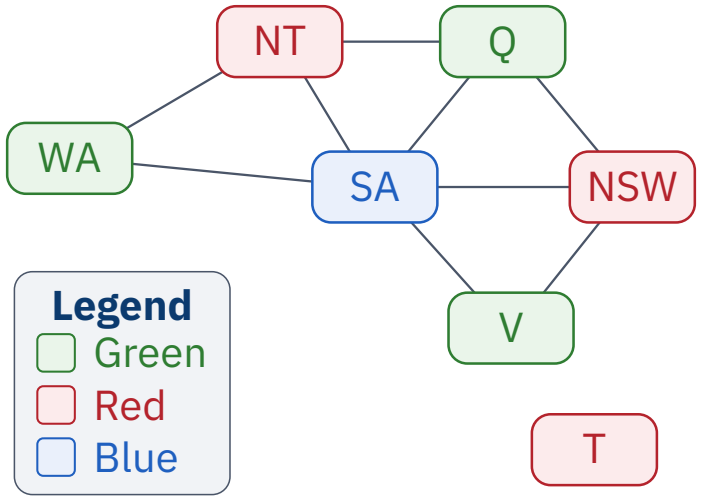
Worked by Hand: The Triangle Map

Three regions A, B, C, every pair adjacent, three colours available.

- **Variables:** A, B, C
- **Domain:** $\{0, 1, 2\}$ each (red/green/blue)
- **Constraints:** $A \neq B$, $B \neq C$, $A \neq C$
- **One solution:** $A{=}0, B{=}1, C{=}2$ – complete *and* satisfies all three constraints

UNIT 1 · CSP BASICS

From Three Regions to Seven: The Australia Map



One node per region-variable, one edge per ≠ constraint – a real, solver-verified 3-colouring (Unit 3/4 build this model).

UNIT 1 · CSP BASICS

CSP vs. COP vs. SAT

- **COP**: same triple, plus an objective function – "find the *best*" feasible solution, not just any
- **SAT**: the special case where every domain is $\{0, 1\}$ and every constraint is a logical clause
- This course stays in pure-CSP territory this week; COP returns in Week 8

3 · Participation

Quick Check: CSP or COP?

A small conference has 3 talks and 2 slots (morning/afternoon). T_1 and T_2 share an author and cannot share a slot.

- What are the variables? A plausible domain? The one constraint?
- Does finding *any* valid schedule make this a CSP or a COP?

One variable per talk, domain $\{\text{morning}, \text{afternoon}\}$, constraint $\text{value}(T_1) \neq \text{value}(T_2)$ – a pure CSP, since any valid schedule is accepted.

UNIT 2

Modeling as a Skill

2 · Content Same Problem, Different Models

Two students model the same magic square puzzle for homework.

- One’s solver finds every solution in under a second
- The other’s is still running five minutes later
- Same problem, same computer – the difference was made before either typed a single line of solver code

UNIT 2 · MODELING SKILL

Viewpoints: More Than One Valid Way to See a Problem

4-queens, two competing viewpoints:

- **A – one variable per row:** 4 variables, domain $\{1, 2, 3, 4\}$ (compact; row-uniqueness comes free)
- **B – one Boolean per square:** 16 variables, domain $\{0, 1\}$ (bulkier; row-uniqueness needs an explicit constraint)

Neither is “wrong” – they trade off variable count against how naturally later constraints fall out of the choice.

UNIT 2 · MODELING SKILL

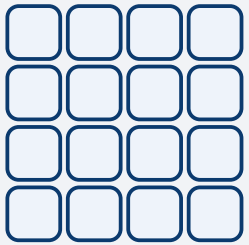
Two Viewpoints, Side by Side

Viewpoint A — one variable per row

Q1 domain = {1,2,3,4}
Q2 domain = {1,2,3,4}
Q3 domain = {1,2,3,4}
Q4 domain = {1,2,3,4}

4 variables, 4 possible values each

Viewpoint B — one Boolean per square



16 variables, 2 possible values each (0/1)

Same problem, same solutions — different search space.

Same problem, same solutions – different search space.

UNIT 2 · MODELING SKILL

Auxiliary Variables: Bookkeeping, Not New Unknowns

Definition: Auxiliary variable

A variable added to a model to make some constraint easier to state, without changing what counts as a solution to the original problem.

Analogy

A subtotal cell on a budget spreadsheet is not a new expense – it is a bookkeeping variable that makes a later rule (“subtotal must stay under the cap”) easy to check without re-adding every expense cell.

UNIT 2 · MODELING SKILL

Implied Constraints: Free Information for the Solver

Definition: Implied (redundant) constraint

A constraint that logically follows from constraints already in a model; adding it changes no solution but may help a solver prune faster.

Case Study 1: A permutation of 1–4

`AllDifferent` on four variables over $\{1, 2, 3, 4\}$ already forces their sum to 10 in every solution – stating `sum(vars) == 10` changes nothing about which assignments are solutions, but can help a solver prune sooner.

UNIT 2 · MODELING SKILL

Why This Matters Before You Write Solver Code

- Viewpoint, auxiliary variables, implied constraints – three choices made before any solver call
- Two people can hand a solver “correct” but very differently-behaving models of the same problem
- Weeks 2–6 make that difference measurable, in propagation strength and running time, on this week’s own benchmark problems

UNIT 3

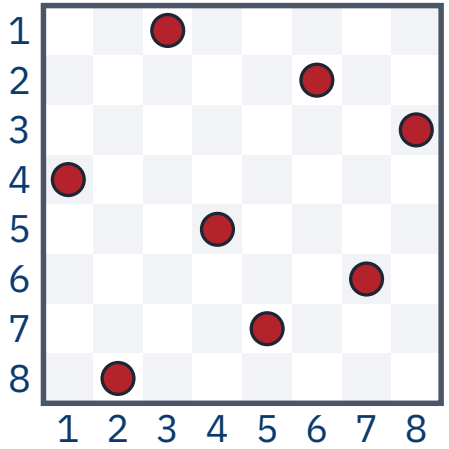
Classic Problems as Models

2 · Content N-Queens: One Queen per Row

- **Variables:** $Q_1, \ldots, Q_N$, one per row
- **Domain:** $\{1, \ldots, N\}$ (the column)
- **Constraints:** `AllDifferent`$(Q_1, \ldots, Q_N)$ (columns) plus, for all $i \neq j$: $|Q_i - Q_j| \neq |i - j|$ (diagonals)

Common mistake: `AllDifferent` alone only rules out shared columns – rows are already free by construction, but diagonals need this extra condition.

One Real Solution: 8-Queens



One of 92 solutions for 8-queens (Week 1); see Week 3 for how search finds it and Week 2 for how propagation prunes it.

One of 92 solutions – read directly from a genuine solver run, not invented by hand.

Sudoku: One Variable per Cell

- **Variables:** 81 – one per cell
- **Domain:** $\{1, \ldots, 9\}$
- **Constraints:** givens fixed by equality, plus `AllDifferent` on every row, column and 3×3 box – 27 in total

A given is a constraint too – just an equality one, in exactly the same formal sense as `AllDifferent`.

Map Coloring: The Australia Instance

- **Variables:** WA, NT, SA, Q, NSW, V, T – 7 regions
- **Domain:** 3 colours each
- **Constraints:** one ≠ per adjacency edge – 9 in total; `AllDifferent` is not used here, since adjacency is edge-by-edge, not all-pairs
- Tasmania borders no region – zero constraints, still needs a colour

UNIT 3 · CLASSIC PROBLEMS

Magic Square: On Paper This Week

- N^2 cells, domain $\{1, ..., N^2\}$
- `AllDifferent` over all cells, plus one equal-sum constraint per row, per column and per diagonal
- No code this week – Week 3 needs its larger search space to make search heuristics visible

UNIT 4

First Models in CP-SAT

2 · Content Three Building Blocks

- `cp_model.CpModel()` – an empty container for (X, D, C)
- `model.NewIntVar(low, high, name)` – one variable, one domain
- `model.Add(...)` or `model.AddAllDifferent([...])` – a constraint

Every model this semester follows this same recipe – CP-SAT this week, MIP solvers such as Gurobi from Week 7 onward.

UNIT 4 · CP-SAT IN CODE

Code: The Toy Triangle CSP

```
12 region_a = model.NewIntVar(0, 2, "region_a")
13 region_b = model.NewIntVar(0, 2, "region_b")
14 region_c = model.NewIntVar(0, 2, "region_c")
15 return [region_a, region_b, region_c]
```

```
22 model.Add(region_a != region_b)
23 model.Add(region_b != region_c)
24 model.Add(region_a != region_c)
```

Each `NewIntVar(0, 2, ...)` is Unit 1's domain $\{0, 1, 2\}$ written directly into code.

UNIT 4 · CP-SAT IN CODE

Code: Solving and Reading the Result

```
34 solver = cp_model.CpSolver()
35 solver.parameters.num_workers = 1
36 solver.parameters.random_seed = 42
37 status = solver.Solve(model)
38 values = [solver.Value(region) for region in regions]
39 print(f"Status: {solver.StatusName(status)}")
40 print(f"Assignment: region_a={values[0]}, region_b={values[1]}, region_c={values[2]}")
```

Expected Output

```
Status: OPTIMAL
Assignment: region_a=2, region_b=1, region_c=0
```

UNIT 4 · CP-SAT IN CODE

Common Pitfall: `model.Add` Records, It Does Not Evaluate

⚠ Common Pitfall

Writing `region_a != region_b` outside `model.Add(...)` does nothing useful – Python evaluates it against the `IntVar` objects themselves, not their eventual solved values, and throws the result away. It only becomes part of the CSP once wrapped in `model.Add(...)`.

UNIT 4 · CP-SAT IN CODE

Enumerating All Solutions

- `solver.Solve(model)` stops at the first solution it finds
- Set `solver.parameters.enumerate_all_solutions = True` and pass a solution callback to keep searching instead
- `OnSolutionCallback` fires once per complete solution – this is how you *count*, not just find

UNIT 4 · CP-SAT IN CODE

Code: N-Queens in CP-SAT

```
12 queens = [model.NewIntVar(0, n - 1, f"queen_{row}") for row in range(n)]
```

```
16 model.AddAllDifferent(queens)
17 model.AddAllDifferent([queens[i] + i for i in range(n)])
18 model.AddAllDifferent([queens[i] - i for i in range(n)])
```

Three `AllDifferent` calls: one on the columns directly, two more on the columns after the diagonal transformations $Q_i + i$ and $Q_i - i$.

3 · Participation Quick Check: Fix the Missing Constraints

A 6-queens script only has `model.AddAllDifferent(queens)` for columns. Add the two missing diagonal calls, then run it and report the solution count.

`model.AddAllDifferent([queens[i] + i for i in range(6)])` and `model.AddAllDifferent([queens[i] - i for i in range(6)])` – with both added, enumeration reports exactly 4 solutions (OEIS A000170).

UNIT 4 · CP-SAT IN CODE

Real Output: 8-Queens and 6-Queens

Expected Output

```
Status: OPTIMAL
Columns per row (0-indexed) for N=8: [2, 5, 7, 0, 3, 6, 4, 1]
```

Expected Output

```
Number of solutions for N=6: 4
```

Column 2 (0-indexed) on row 1 is the same queen as column 3 on the 1-indexed board you saw earlier – one shift of 1, applied uniformly.

UNIT 4 · CP-SAT IN CODE

Real Output: A Completed Sudoku

Expected Output

```
Status: OPTIMAL
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

Every row, column and 3×3 box is a permutation of 1–9 – exactly what the model's 27 `AllDifferent` constraints demand.

UNIT 4 · CP-SAT IN CODE

Code and Output: Map Coloring

```
35 for region_a, region_b in EDGES:
36     model.Add(colours[region_a] != colours[region_b])
```

Expected Output

```
Status: OPTIMAL
WA: Green
NT: Red
SA: Blue
Q: Green
NSW: Red
V: Green
T: Red
```

Summary

SUMMARY

Summary

Summary

- A CSP is variables + domains + constraints; a solution is a complete, constraint-satisfying assignment
- Modeling is a skill – viewpoint, auxiliary variables and implied constraints change how a solver experiences the *same* problem
- N-queens, Sudoku, map coloring and magic square are all CSPs in disguise – the field's shared benchmark problems
- OR-Tools CP-SAT turns a paper model into a running program in three building blocks: model, variables, constraints

SUMMARY

4 · Assessment

Check Yourself

- One Check Yourself question closes each unit in this week's chapter – answers are collected at the chapter's end, not given immediately
- Furniture-store adjacency (CSP or COP?), why an implied constraint can help without changing the solution set, 4-queens on paper, `Solve` vs. enumeration

SUMMARY

5 · Follow-through

Transfer — Where Else?

- Weeks 2–6 reuse today's exact N-queens, Sudoku and map-coloring models – propagation, then search, then three solvers
- Week 2 asks how much work propagation actually saved you on the models you just built
- Scheduling (Weeks 11–12) and routing (Week 14) start with exactly this choice: viewpoint, auxiliary variables, implied constraints
- Keep `nqueens.py`, `sudoku.py` and `map_coloring.py` – you will open them again

SUMMARY

In the Lab

This week's lab sheet
You build and run all four models yourself, hand-trace one classic problem, and close by writing a complete CP-SAT model from scratch for a problem not already coded for you.

- Next week: constraint propagation – the solver's black box opens
- The same N-queens and map-coloring models, watched shrinking *before* any search happens